

Exercise 8: Multivariate Optimization 3 (SGD)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("gridExtra", "2.3")
```

Consider ordinary linear least squares (without intercept):

$$\min_{\theta} \mathbb{E}_{\mathbf{x}, y} [(\theta^\top \mathbf{x} - y)^2]$$

with $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{x}})$ and $y \mid \mathbf{x} \sim \mathcal{N}(\theta^{*\top} \mathbf{x}, \sigma^2)$. Throughout the coding parts we use the univariate setting $\Sigma_{\mathbf{x}} = 1/4$, $\sigma = 1/10$, $\theta^* = 1/2$. Each coding cell draws fresh samples from this generative model (fresh mini-batches in (c); streaming single samples in (e)).

Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs. The data above (and consequently the SGD trajectories below) differ between languages. Both implementations are correct; only the underlying data differs. Compare *algorithmic behavior* (cloud shape, convergence pattern) rather than expecting

numerically identical θ values.

(a) Unbiasedness of the SGD gradient

Show that

$$\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]] = \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2].$$

Solution.

We compute both expressions and compare the results.

$$\begin{aligned}\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]] &= \mathbb{E}_{\mathbf{x},y}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}y] \\ &= \mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}y] \\ &= \mathbb{E}_{\mathbf{x}}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}\mathbf{x}^{\top}\theta^*] \\ &= 2\Sigma_{\mathbf{x}}(\theta - \theta^*).\end{aligned}$$

$$\begin{aligned}\nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2] &= \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta - 2\theta^{\top}\mathbf{x}y + y^2] \\ &= \nabla_{\theta}\mathbb{E}_{\mathbf{x}}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta] - \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[2\theta^{\top}\mathbf{x}y] + \nabla_{\theta}\mathbb{E}_y[y^2] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}\mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[\theta^{\top}\mathbf{x}y] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}\mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta^*] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}(\theta^{\top}\Sigma_{\mathbf{x}}\theta^*) \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\Sigma_{\mathbf{x}}\theta^* \\ &= 2\Sigma_{\mathbf{x}}(\theta - \theta^*).\end{aligned}$$

(b) Implication for SGD

Interpret (a) in terms of stochastic gradient descent.

Solution.

We can estimate $\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]]$ without bias via SGD, since we have access to realizations of the gradients $\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]$. From (a), it follows that this estimate is also an unbiased estimate of the gradient of our objective function $\nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2]$. Hence, SGD can be successfully applied in this situation.

(c) Gradient variance simulation

In the univariate setting $\Sigma_x = 1/4$, $\sigma = 1/10$, $\theta^* = 1/2$, with data of size 10,000:

Plot the **confusion** (variance of the SGD gradient around the true full-batch gradient) for $\theta \in \{0, 0.05, 0.10, \dots, 1.00\}$. For each θ draw 200 independent batches and plot the per-batch confusion. Repeat for batch sizes $m = 100$ and $m = 1,000$.

Solution.

R

```
library(ggplot2)
library(gridExtra)

set.seed(123)
sigma_x <- 0.5 # Sigma_x = 1/4, so sd = 1/2
sigma_y <- 0.1
theta_star <- 0.5

# data of size n (drawn once; each confusion below re-samples a fresh batch)
n <- 10000
x <- sort(rnorm(n, sd = sigma_x))
y <- theta_star * x + rnorm(n, sd = sigma_y)

# confusion: mean squared deviation of the per-sample gradients from the
# population gradient 2 * sigma_x^2 * (theta - theta_star)
compute_conf <- function(theta, m) {
  x <- rnorm(m, sd = sigma_x)
  y <- theta_star * x + rnorm(m, sd = sigma_y)
  mean((2 * (x * x * theta - y * x) - 2 * sigma_x^2 * (theta - theta_star))^2)
}

thetas <- seq(0, 1, length.out = 21)
reps <- 200

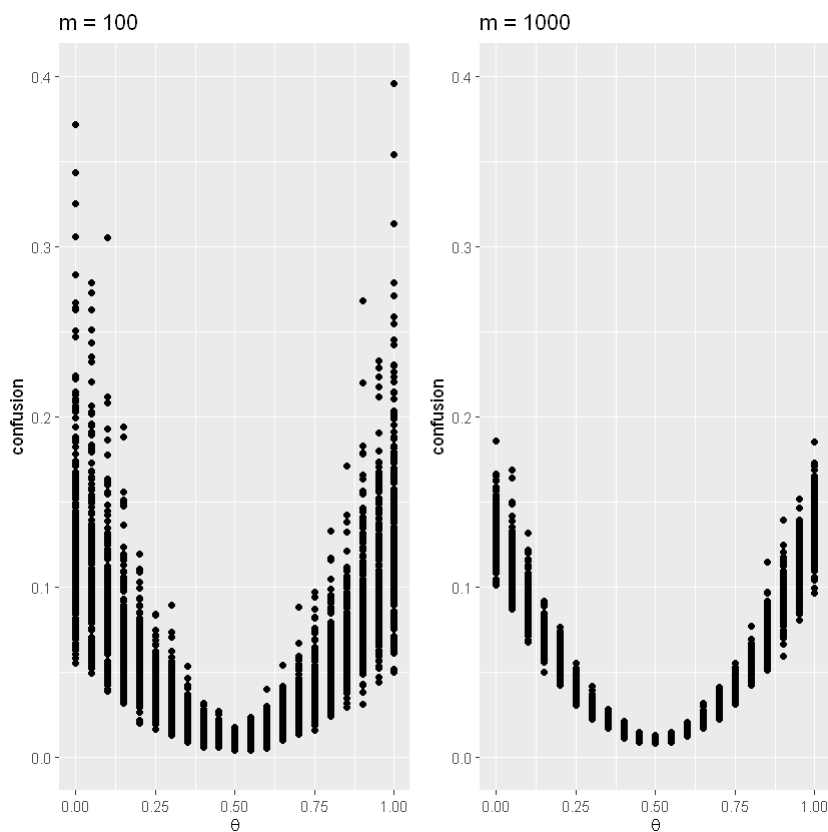
run_grid <- function(m) {
  confs <- c()
  for (i in 1:reps)
    for (theta in thetas) confs <- c(confs, compute_conf(theta, m))
  data.frame(theta = rep(thetas, reps), conf = confs)
}
```

```

df_100  <- run_grid(100)
df_1000 <- run_grid(1000)

p1 <- ggplot(df_100, aes(theta, conf)) + geom_point() +
  ylim(0, 0.4) + xlab(expression(theta)) +
  ylab("confusion") + ggtitle("m = 100")
p2 <- ggplot(df_1000, aes(theta, conf)) + geom_point() +
  ylim(0, 0.4) + xlab(expression(theta)) +
  ylab("confusion") + ggtitle("m = 1000")
grid.arrange(p1, p2, ncol = 2)

```



Python

```

import numpy as np
import matplotlib.pyplot as plt

```

```

np.random.seed(123)
sigma_x = 0.5 # Sigma_x = 1/4, so sd = 1/2
sigma_y = 0.1
theta_star = 0.5

# data of size n (drawn once; each confusion below re-samples a fresh batch)
n = 10000
x = np.sort(np.random.normal(0, sigma_x, n))
y = theta_star * x + np.random.normal(0, sigma_y, n)

def compute_conf(theta, m):
    # confusion: mean squared deviation of the per-sample gradients from the
    # population gradient  $2 * \sigma_x^2 * (\theta - \theta_{star})$ 
    x = np.random.normal(0, sigma_x, m)
    y = theta_star * x + np.random.normal(0, sigma_y, m)
    per_sample = 2 * (x * x * theta - y * x)
    pop_grad = 2 * sigma_x ** 2 * (theta - theta_star)
    return float(np.mean((per_sample - pop_grad) ** 2))

thetas = np.linspace(0, 1, 21)
reps = 200

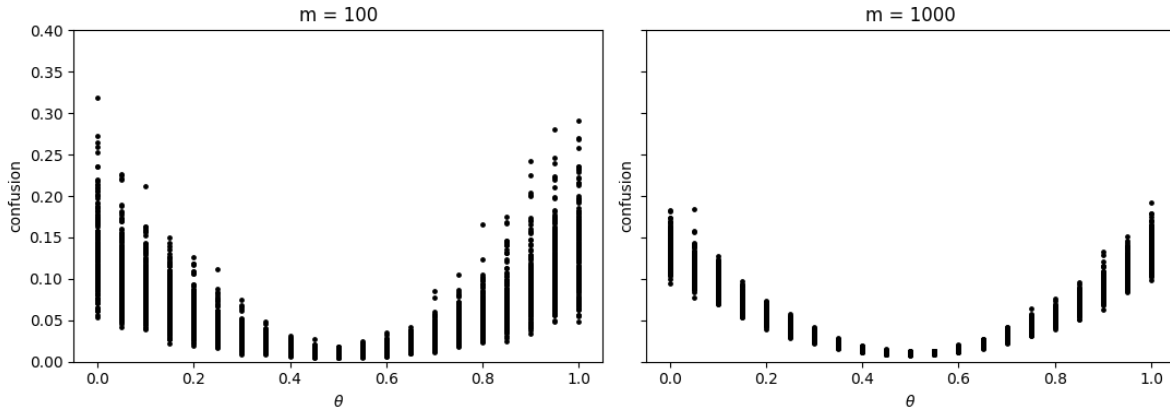
def run_grid(m):
    out = np.empty(reps * len(thetas))
    k = 0
    for _ in range(reps):
        for th in thetas:
            out[k] = compute_conf(th, m); k += 1
    return out

vals_100 = run_grid(100)
vals_1000 = run_grid(1000)

fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for ax, vals, m_lbl in zip(axes, (vals_100, vals_1000), (100, 1000)):
    ax.scatter(np.tile(thetas, reps), vals, s=6, color="black")
    ax.set_ylim(0, 0.4)
    ax.set_xlabel(r"$\theta$"); ax.set_ylabel("confusion")

```

```
ax.set_title(f"m = {m_lbl}")
plt.tight_layout()
plt.show()
```



For each θ and batch size m we draw a fresh batch and measure the *confusion*: the mean squared deviation of the per-sample gradients $2(x_i^2\theta - y_ix_i)$ from the population gradient $g(\theta) = 2\sigma_x^2(\theta - \theta^*)$. This estimates the sample-to-sample variance of the gradient at θ , which the larger batch $m = 1000$ pins down more tightly than $m = 100$.

(d) Reading the simulation

What do you observe in (c)?

Solution.

Qualitatively, we observe for both settings that the mean and the variance of the confusion rise symmetrically around θ^* . As expected, the mean and the variance of the confusion are smaller for the larger batch size $m = 1000$ than for $m = 100$.

(e) SGD vs population GD

On the same setup, run SGD with **batch size 1**, step size $\alpha = 0.3$, starting at $\theta = 0$, for 20 iterations. Repeat the SGD run 200 times and overlay all trajectories. Compare with **population GD** (same step size, same start) using the analytical gradient $g(\theta) = 2\sigma_x^2(\theta - \theta^*)$.

Solution.

R

```
set.seed(123)
alpha <- 0.3
n_iters <- 20
n_runs <- 200
m <- 1

# SGD with batch size 1
sgd_traces <- matrix(NA, n_runs, n_iters)
for (j in 1:n_runs) {
  theta <- 0
  for (i in 1:n_iters) {
    x <- rnorm(m, sd = sigma_x)
    y <- theta_star * x + rnorm(m, sd = sigma_y)
    theta <- theta - alpha * mean(2 * (x * x * theta - y * x))
    sgd_traces[j, i] <- theta
  }
}

# Population GD using the analytical gradient
#  $2 * \sigma_x^2 * (\theta - \theta_{\text{star}})$ 
theta <- 0
gd_trace <- theta
for (i in 1:n_iters) {
  theta <- theta - alpha * 2 * sigma_x^2 * (theta - theta_star)
  gd_trace <- c(gd_trace, theta)
}

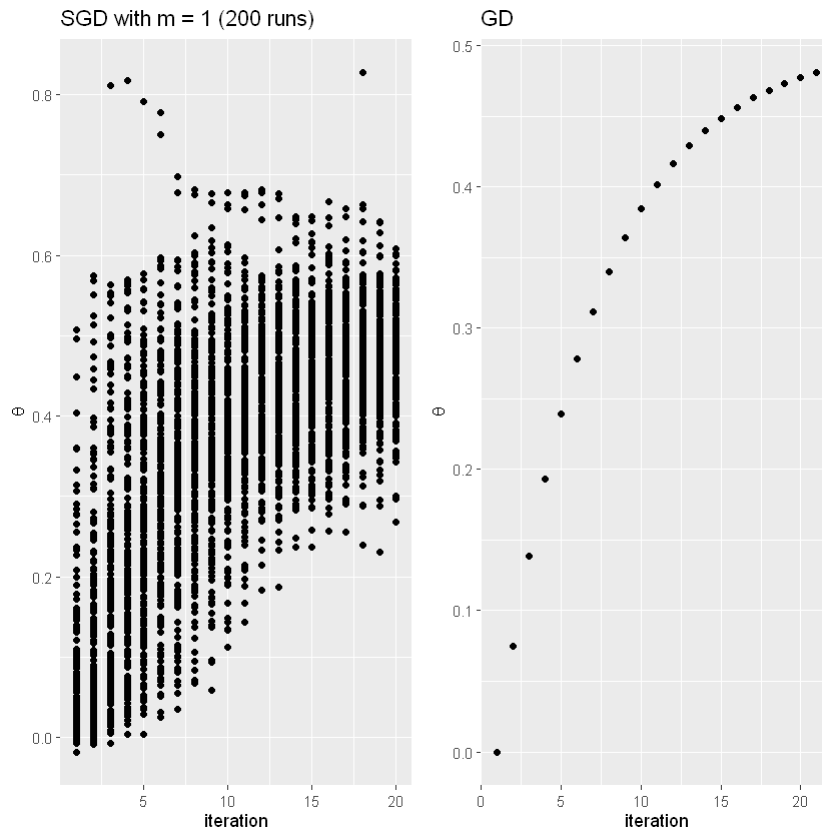
df_sgd <- data.frame(it = rep(1:n_iters, each = n_runs),
                     theta = as.numeric(sgd_traces))
df_gd <- data.frame(it = seq_along(gd_trace), theta = gd_trace)

p_sgd <- ggplot(df_sgd, aes(it, theta)) + geom_point() +
  ylab(expression(theta)) + xlab("iteration") +
  ggtitle("SGD with m = 1 (200 runs)")
p_gd <- ggplot(df_gd, aes(it, theta)) + geom_point() +
  ylab(expression(theta)) + xlab("iteration") + ggtitle("GD")
grid.arrange(p_sgd, p_gd, ncol = 2)

cat(sprintf(paste0(
  "After 20 iters:  SGD mean = %+.4f, sd = %.4f;  ",
```

```
"GD theta = %+4f  (theta* = %.4f)\n"),
    mean(sgd_traces[, n_iters]), sd(sgd_traces[, n_iters]),
    gd_trace[length(gd_trace)], theta_star))
```

After 20 iters: SGD mean = +0.4647, sd = 0.0626; GD theta = +0.4806 (theta* = 0.5000)



Python

```
np.random.seed(123)
alpha = 0.3
n_iters = 20
n_runs = 200
m = 1

# SGD with batch size 1
```



```

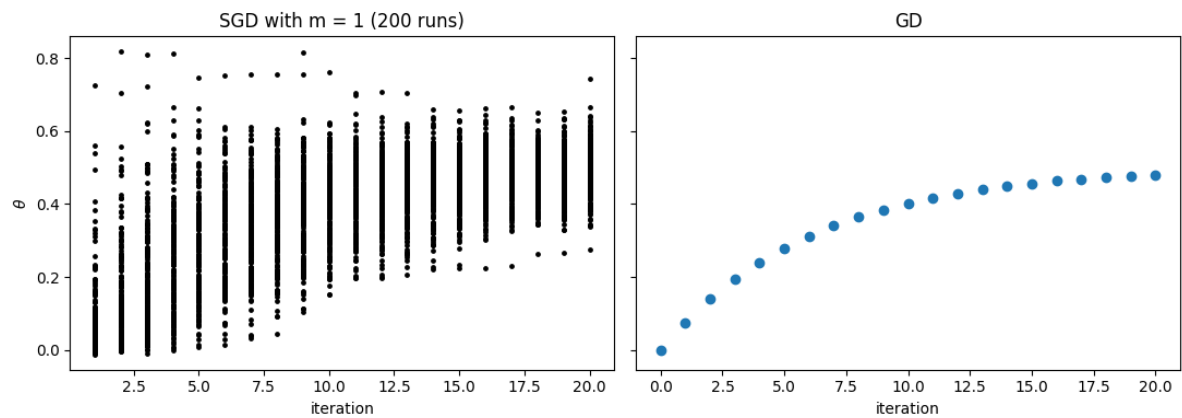
sgd_traces = np.empty((n_runs, n_iters))
for j in range(n_runs):
    theta = 0.0
    for i in range(n_iters):
        x = np.random.normal(0, sigma_x, m)
        y = theta_star * x + np.random.normal(0, sigma_y, m)
        theta = theta - alpha * np.mean(2 * (x * x * theta - y * x))
        sgd_traces[j, i] = theta

# Population GD using the analytical gradient
#  $2 * \sigma_x^2 * (\theta - \theta_{\text{star}})$ 
theta = 0.0
gd_trace = [theta]
for i in range(n_iters):
    theta = theta - alpha * 2 * sigma_x ** 2 * (theta - theta_star)
    gd_trace.append(theta)

fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
sgd_it = np.arange(1, n_iters + 1)
for j in range(n_runs):
    axes[0].scatter(sgd_it, sgd_traces[j], s=6, color="black")
axes[0].set_xlabel("iteration"); axes[0].set_ylabel(r"$\theta$")
axes[0].set_title("SGD with m = 1 (200 runs)")
axes[1].scatter(np.arange(len(gd_trace)), gd_trace)
axes[1].set_xlabel("iteration"); axes[1].set_title("GD")
plt.tight_layout()
plt.show()

print(f"After 20 iters:  SGD mean = {sgd_traces[:, -1].mean():+.4f}, "
      f"sd = {sgd_traces[:, -1].std():.4f};  "
      f"GD theta = {gd_trace[-1]:+.4f}  (theta* = {theta_star:.4f})")

```



After 20 iters: SGD mean = +0.4862, sd = 0.0688; GD theta = +0.4806 ($\theta^* = 0.5000$)